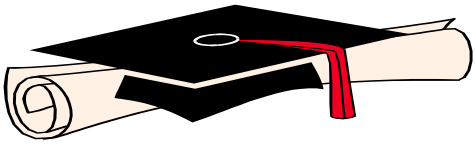
EPC

Elementary Programming with C

Semester: 1	**The Exam Code:** 02
Duration: 90 minutes	**Total Marks:** 20



Practical Examination Paper

Do not write on this question paper and return it to the Invigilator after the examination.

C Programming Exam: Inventory Management System

Total Points: 20
Duration: 90 minutes

Instructions

- Ensure your code is modular, well-structured, and includes comments explaining key logic.
- Use only standard C libraries.
- Validate user inputs and handle errors appropriately.

Question 1: Basic Input and Output (3 Points)

Task:
Write a program that calculates the total cost of items purchased.

Instructions:

1. Prompt the user to input the number of items purchased.
2. Use a loop to input the price of each item and calculate the total cost.
3. Display the total cost to the user.

Example Input/Output:

```
Enter the number of items: 3
Enter price of item 1: 10
Enter price of item 2: 20
Enter price of item 3: 30
Total cost: $60
```

Points Distribution:

- Correct input handling for the number of items: **1 Point**
- Loop and calculation logic: **1.5 Points**
- Displaying the total cost: **0.5 Points**

Question 2: Complex Expression Evaluation (4 Points)

Task:
Write a program that evaluates a complex mathematical expression using user inputs.

Expression to Evaluate:

```
result = (a + b)^c - (d / e)
```

Instructions:

- Prompt the user to input five values: `a`, `b`, `c`, `d`, and `e`.
- Validate inputs to ensure `e != 0`.
- Use the `pow` function from `<math.h>` to calculate `(a + b)^c`.
- Display the result with two decimal points.

Example Input/Output:

```
Enter value for a: 2
Enter value for b: 3
Enter value for c: 2
Enter value for d: 10
Enter value for e: 2
Result: 20.50
```

Points Distribution:

- Input and validation logic: **1 Point**
- Correct implementation of the expression: **2.5 Points**
- Displaying the result: **0.5 Points**

Question 3: Structure and CRUD Operations (6 Points)

Task:
Write a program to manage product inventory using a structure and basic CRUD operations.

Instructions:

- Define a structure `Product` with the following fields:
  - `id` (int): Product ID
  - `name` (char[50]): Product name
  - `quantity` (int): Product quantity

- `price` (float): Product price

Implement the following functionalities:

1. **Add Product:**
   - Allow the user to add up to 5 products.
   - Validate inputs (e.g., name cannot be empty, quantity > 0, price > 0).
2. **View Products:**
   - Display all products with their details.
3. **Delete Product:**
   - Remove a product by ID.

Example Menu and Output:

```
Menu:

1. Add Product

2. View Products

3. Delete Product

4. Exit

Your choice: 1

Enter Product ID: 101

Enter Product Name: Laptop

Enter Quantity: 10

Enter Price: 800



Your choice: 2

ID: 101, Name: Laptop, Quantity: 10, Price: $800.00
```

Points Distribution:

- Structure definition: **1 Point**
- Add Product functionality: **2 Points**
- View, Delete functionalities: **3 Points**

Question 4: Complex Expression Evaluation (4 Points)

Task:
Write a program that evaluates a complex mathematical expression using user inputs.

Expression to Evaluate:

```
result = (a + b)^c - (d / e)
```

Instructions:

- Prompt the user to input five values: `a`, `b`, `c`, `d`, and `e`.
- Validate inputs to ensure `e != 0`.
- Use the `pow` function from `<math.h>` to calculate `(a + b)^c`.
- Display the result with two decimal points.

Example Input/Output:

```
Enter value for a: 2

Enter value for b: 3

Enter value for c: 2

Enter value for d: 10

Enter value for e: 2

Result: 20.50
```

Points Distribution:

- Input and validation logic: **1 Point**
- Correct implementation of the expression: **2.5 Points**
- Displaying the result: **0.5 Points**

Question 5: Dynamic Memory Allocation and Sorting (3 Points)

Task:
Write a program to dynamically allocate memory for an array of integers and sort the array in descending order.

Instructions:

1. Input the size of the array and dynamically allocate memory.
2. Input elements of the array from the user.
3. Use a sorting algorithm (e.g., bubble sort) to sort the array in descending order.
4. Display the sorted array.

Example Input/Output:

```
Enter the number of elements: 5
Enter 5 integers: 3 1 4 1 5
Sorted array: 5 4 3 1 1
```

Points Distribution:

- Correct use of dynamic memory allocation: **1 Point**
- Implementing descending sort logic: **1.5 Points**
- Displaying the sorted array: **0.5 Points**

Submission Instructions

1. Submit all `.c` source code files.
2. Include comments explaining your logic for each question.
3. Test your code thoroughly with edge cases and valid inputs.
4. Package all files in a zipped folder named `C_Exam_<YourName>.zip`.